Autonomous Agents for the Deliveroo.js Environment

Autonomous Software Agents - Final Project Report
University of Trento,

Riderless

Abstract

This report describes the design and implementation of an autonomous agent for the **Deliveroo.js** game environment, a grid-world simulation in which agents compete to pick up parcels and deliver them to designated drop zones. The agent follows a *Belief-Desire-Intention* (BDI) architecture written in TypeScript. Decisions such as: which parcel to collect, where to deliver, and where to explore are driven by a utility function that accounts for parcel reward, decay rate, distance heuristics, and competitive pressure from other agents. Path-finding is handled by a BFS-based planner that can optionally be upgraded to an optimal PDDL solver. A secondary LLM interface allows the agent to interpret natural-language commands, translating them into BDI desires, and to communicate with another agent to cooperate effectively and maximize points.

1 Introduction

1.1 Context and Motivation

Deliveroo.js is a browser-based, multi-agent grid world developed at the University of Trento for the Autonomous Software Agents course. Agents connect to a server over a WebSocket API, receive perceptual updates (parcels, tiles, other agents), and must act continuously to maximize their score. Additionally, the agent can process textual instructions to earn extra points, balancing these requests with its primary tasks whenever possible. This setup provides an ideal testbed to explore how symbolic planning (PDDL) and LLM-based reasoning can complement a classical BDI loop.

1.2 Project Goals

1. **BDI Agent Architecture (Challenge 1):** Design and implement a deliberative agent capable of sensing the environment, dynamically revising beliefs, and managing intentions to collect and deliver parcels efficiently.
2. **Symbolic Planning Integration:** Integrate an asynchronous PDDL planner to support optimal path-finding and navigate complex grid topologies.
3. **LLM Natural-Language Interface (Challenge 2):** Develop an LLM pipeline to handle atomic operator requests, dynamically adapt individual game strategies, and parse complex operational constraints.
4. **Multi-Agent Coordination:** Implement a collaborative strategy allowing the LLM-enhanced agent, the core BDI agent, and the external mission-agent to communicate,

exchange environment data, and coordinate actions toward shared goals.

2 System Architecture

2.1 Overview

The system operates on a reactive sense-deliberate-act cycle, where each server-side sensing event drives a new iteration of the BDI loop. Figure 1 sketches the high-level data flow.

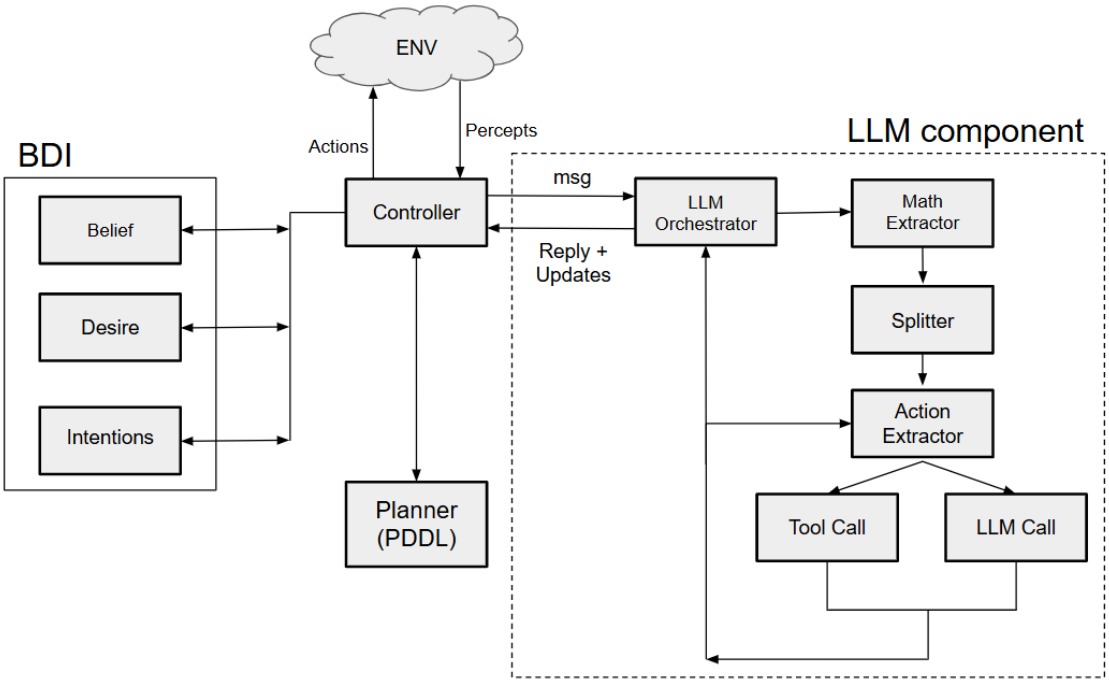

Figure 1: Agent Architecture

The system comprises three main subsystems:

- **BDI**: maintains world beliefs, generates desires, and commits to intentions.
- **Planning Component (PDDL)**: computes movement paths.
- **LLM Component**: interprets natural-language instructions and leverages a dedicated suite of **deterministic tools** (e.g., for mathematical evaluations) to inject updates into the BDI loop.

The entry point `main.ts` wires all components together, handles the WebSocket events, and runs the main BDI step function.

2.2 BDI Agent Model

The BDI layer is split across three files.

Beliefs (`BDI/Belief.ts`). The belief layer implements the **belief update** step of the BDI architecture, maintaining an internal representation of the environment under partial observability. During initialization, the agent builds the static map and stores its initial

position. Subsequent sensing events update the dynamic state, including parcels, crates, and other agents.

Beliefs are stored in the `World` as coordinate-indexed maps. New observations update the current state, while entities that leave the observation range are retained briefly through timestamp-based lifespans, providing limited short-term memory.

The belief update process also derives task-relevant information from observations, such as tracking opponent movements and maintaining obstacle information.

Finally, **belief revision** allows the agent to avoid cells considered temporarily blocked, either due to dynamic obstacles detected between sensing cycles or cells marked as inaccessible from LLM-interpreted instructions. As a result, the belief state combines current observations, persistent map knowledge, and short-term memory to support intention selection and execution.

Desires (`BDI/Desire.ts`). A Desire is represented as a tuple $\langle \text{type}, x, y, u \rangle$, where $u \in \mathbb{R}$ denotes its utility score. The agent generates three internal desire types:

- `go_pickup`: generated for each visible, uncarried parcel; its utility (Sec. 2.5.1) trades off reward, distance, and competitive risk.
- `go_delivery`: generated while carrying at least one parcel; its utility (Sec. 2.5.2) estimates the remaining reward net of decay and opponent occupancy at delivery tiles.
- `explore`: fallback behavior guiding the agent toward unvisited, high-density spawn regions (Sec. 2.5.3); multiple candidates are generated to allow switching if a path becomes unreachable.

Intentions (`BDI/Intentions.ts`). The `reviseIntention` function implements the agent's **intention revision** policy, which is executed at every BDI step. Based on the updated beliefs and a list of desires sorted by utility, this function determines whether the agent should maintain its current commitment or switch to a better candidate.

The control flow handles two main scenarios. If no intention is currently active, the agent automatically commits to the highest-utility desire. Conversely, if an intention is already active, its ongoing *validity* is verified against the current world state. Specifically, delivery intentions remain valid only while the agent is carrying at least one parcel, whereas pickup intentions are kept only as long as the target parcel is still available on the map. Exploration intentions, on the other hand, are always considered valid and act as a fallback behavior when no other tasks can be performed. If the active intention fails its respective validity check, it is immediately discarded and replaced by the best available desire. Intentions can be **blocked** after a fixed number of failures and the agent automatically switch intention to the next best one.

The BDI intention system is extended to support **LLM-injected missions**. Missions extracted from natural-language instructions, such as navigation goals (`go_to`), forced deliveries (`go_delivery`), synchronized waits (`wait_row`), or rendezvous points, are stored in an ordered queue (`llmPendingMissions`) and promoted one at a time to an active slot (`llmActiveMission`). These missions are assigned a high utility value, allowing them to preempt lower-priority autonomous desires.

By contrast, stack constraints and delivery bonuses do not create new intentions. Instead, they modify the utility computation of existing desires, enabling external instructions to influence the agent's preference ordering while preserving the standard revision process. Once an LLM mission is completed, it is removed and the agent resumes its autonomous behavior.

2.3 Planning Component

The planning component enables the agent to compute a path toward a target tile. To evaluate the performance and efficiency of the planning subsystem, we implement a Breadth-First Search (BFS) algorithm as a baseline for shortest-path calculation and compare it against a classical automated planning approach using PDDL.

The PDDL-based module leverages the *Fast Downward* planner, configuring it to use the A^* search algorithm to determine the optimal path. Within this formalization, the planning domain defines the available actions as the movement capabilities of the agent, while the state space represents the grid tiles of the environment.

Furthermore, the PDDL domain is designed to support crate manipulation in single-agent scenarios, enabling the agent to plan crate movements to navigate more complex environments. However, crate handling in cooperative multi-agent settings is not currently supported.

2.4 LLM Component

The LLM component processes textual user inputs and extracts the structured information required to update the BDI agent's state. For this purpose, we selected the *Llama-3.3-70B* model. To optimize performance and context window efficiency, we condition the model's short-term memory using concise system prompts paired with tailored *few-shot examples* for each specific task.

The LLM component is designed to handle and execute a variety of tasks, which can be categorized into three levels of complexity:

- **Atomic Special Missions (Level 1):** Simple, self-contained tasks that either produce a direct reply or inject a single transient update into the BDI loop. These include mathematical computations (`calculate`), time zone queries (`get_current_time`), position reports (`get_my_position`), common-knowledge questions (`common_knowledge`), and tile-level navigation or avoidance goals (`generate_desire`).
- **Intermediate Persistent Missions (Level 2):** Non-atomic, match-wide rules that remain active for the entire run and are maintained in dedicated runtime data structures. These include directional delivery constraints that restrict the agent to specific drop-off zones (`generate_delivery_constraint`); reward-multiplier rules conditioned on the parcel count or total carried score at delivery time (`generate_stack_constraint`); and tile-specific delivery bonuses or blocks (`generate_desire` with `go_delivery` or `avoid_delivery` sub-actions). The corresponding state is stored in dedicated maps and lists (`llmDeliveryBonusTiles`, `llmStackConstraints`, `llmBlockedDeliveryTiles`) that are consulted at every BDI step to bias or override autonomous utility scoring.

- **Coordination or Communication Missions (Level 3):** High-complexity tasks that require explicit peer-to-peer communication between the two agents (`multi_agent_command`). These cover four sub-commands: *rendezvous* at an explicit grid position within a configurable maximum distance; *wait_ row* (red-light) to stop both agents on a row of specified parity; *resume* (green-light) to lift a wait hold; and *parcel_ handoff*, where one agent picks up a parcel and meets the other at a BFS-computed meeting tile so the latter can deliver it for a bonus reward.

The core logic is managed by an orchestrator through the *processMessage* function. The message is first cleaned by a *mathExtractor* call, which identifies nested arithmetic expressions, replaces them with placeholders, and resolves them through a deterministic *calculate* tool rather than risking LLM hallucination on numeric results. A *splitter* call then divides concatenated requests into independent sub-goals (e.g., separating a navigation request from an arithmetic question). Each sub-goal is classified by the orchestrator into one of eight action labels (`calculate`, `get_current_time`, `get_my_position`, `common_knowledge`, `generate_desire`, `generate_delivery_constraint`, `generate_stack_constraint`, `multi_agent_command`), and a dedicated extractor then retrieves its structured parameters and populates the corresponding field of the `LLMUpdate` object: direct-reply actions append text to the response, while BDI-update actions populate typed fields such as `goToTiles`, `deliveryConstraints`, `stackConstraints`, or `multiAgentCommand`.

The `generate_desire` action deserves special attention: its extractor produces one of four sub-actions — `go_to`, `avoid`, `go_delivery`, or `avoid_delivery` — cleanly separating movement goals from delivery-zone preferences without requiring separate orchestrator labels.

The orchestrator aggregates the results, returning both a textual reply and the populated `LLMUpdate` object to be injected into the agent's state. The multi-call pipeline introduces minor latency, but this is acceptable given the sporadic, non-real-time nature of the requests, and the modular design allows each component to be evaluated and tuned independently.

The agent's deterministic **tools** module exposes three functions: **calculate**, a sandboxed expression evaluator that whitelists allowed syntax instead of using a global *eval*, supporting standard *Math* functions and constants; **getCurrentTime**, which resolves a city to its timezone via *city-timezones* and formats the result with *Intl.DateTimeFormat*; and **getMyPosition**, which returns the agent's current coordinates.

All calculated answers and structured updates are also forwarded to the BDI agent, ensuring it has the data needed to execute and claim the points for the completed request.

2.5 Utility Functions

The utility functions define the heuristics used by the agent to rank candidate desires. Each utility combines the expected reward of an action with its travel cost and possible risk factors.

2.5.1 Pick up utility

For a `go_pickup` desire targeting parcel p at position $\mathbf{q}$, the utility is computed as the sum of two terms: the expected value of the parcels already carried by the agent and the

expected value of the new parcel. The final score is then reduced when opponent agents are close to the pickup location:

$$u_{\text{pickup}}(p) = (V_{\text{carry}}(\mathbf{q}) + V_{\text{parcel}}(p, \mathbf{q})) \cdot P_{\text{risk}}(\mathbf{q}) \tag{1}$$

where

$$V_{\text{carry}}(\mathbf{q}) = R_{\text{carry}} - n \cdot \delta \cdot d(\mathbf{a}, \mathbf{q}) \tag{2}$$

is the estimated remaining value of the n parcels already carried (R_{carry}) when reaching the pickup position $\mathbf{q}$ from the current agent position $\mathbf{a}$, penalizing it based on the decay rate δ. This prevents the agent from continuously collecting parcels without ever delivering them, which is especially useful in scenarios characterized by a high abundance of parcels and a low density of competing agents. Then,

$$V_{\text{parcel}}(p, \mathbf{q}) = r_p - \delta \cdot d(\mathbf{q}, \mathbf{d}) \tag{3}$$

is the estimated net value of the new parcel, calculated by subtracting the expected decay incurred during the transit from its position $\mathbf{q}$ to the nearest delivery tile $\mathbf{d}$ from its base reward r_p. The final term, $P_{\text{risk}}(\mathbf{q})$, represents an environmental safety coefficient bounded between 0 and 1 that penalizes the target's utility based on the proximity of threats modeling the operational risk of the parcel being intercepted or stolen before the agent can reach the coordinates $\mathbf{q}$.

2.5.2 Delivery utility

For a `go_delivery` desire targeting delivery tile $\mathbf{d}$, the utility estimates the remaining value of the currently carried parcels after the trip:

$$u_{\text{delivery}}(\mathbf{d}) = (R_{\text{carry}} - n \cdot \delta \cdot d(\mathbf{a}, \mathbf{d})) \cdot P_{\text{risk}}(\mathbf{d}) \tag{4}$$

where R_{carry} is the current reward of the carried items, which decays at a rate of δ for each of the n parcels over the distance $d(\mathbf{a}, \mathbf{d})$ from the agent's position $\mathbf{a}$ to the delivery tile.

The penalty term $P_{\text{risk}}(\mathbf{d})$ handles stationary threats. The agent targets the three closest hazard-free delivery tiles ($P_{\text{risk}} = 1$). If all available delivery positions are blocked by stationary enemies, the agent falls back to these targets but scales their utility by $P_{\text{risk}} = 0.1$ to penalize the high-risk action.

2.5.3 Explore utility

For an `explore` desire targeting tile s, the utility rewards tiles that have not been visited recently, balanced against the distance required to reach them:

$$u_{\text{explore}}(s) = \frac{T_{\text{unvisited}}(s)}{d(\mathbf{a}, s) + 1} - C_{\text{travel}}(s) \tag{5}$$

where $T_{\text{unvisited}}(s)$ represents the time elapsed since the tile s was last visited, and $C_{\text{travel}}(s) = K_{\text{penalize}} \cdot \delta \cdot d(\mathbf{a}, s)$ defines the travel cost, heavily penalizing distant targets based on a tuning constant K_{penalize} and the map decay rate δ. This score is further adjusted by environmental multipliers to favor high-density parcel spawns while avoiding areas with high opponent pressure.

2.6 Main Strategies

Stuck and bounce recovery. The agent implements two deterministic recovery mechanisms to prevent deadlocks and repetitive, ineffective behaviors. First, if the pathfinder fails to compute a valid route for the same intention across `STUCK_THRESHOLD` consecutive reasoning cycles, the target intention is temporarily blacklisted for `INTENTION_BLOCK_MS`, forcing the agent to switch to an alternative candidate desire. Second, the agent monitors its recent coordinate history to detect cyclic oscillations, such as $A \to B \to A \to B$ patterns. When such a bounce loop is identified, the involved cells are temporarily blocked, the current path is discarded, and a new route is forced.

Collision avoidance. To ensure smooth navigation, the agent proactively predicts potential collisions by estimating the immediate future position of nearby opponents based on their last observed moving direction. If the agent's next planned step coincides with a predicted opponent position, the current path is cleared and immediately recomputed to avoid a confrontation. Furthermore, opponents that remain stationary across multiple consecutive sensing updates are dynamically registered as static obstacles within the pathfinding grid, allowing the agent to route around them efficiently.

2.7 Communication Mechanisms

The multi-agent architecture follows a **master/slave** deployment model. One agent is designated as the *master* (`IS_MASTER=true`) and the other as the *slave*. The master agent is the sole point of entry for all natural-language instructions: it runs the full LLM pipeline and, immediately after each `processMessage` call, forwards the resulting `LLMUpdate` payload to the slave via a `LLM_UPDATE` peer message. Both agents then apply the same structured update to their respective BDI loops, ensuring synchronized belief state without requiring the slave to carry its own LLM client.

Communication relies on the Deliveroo.js peer-to-peer messaging layer (`socket.emitSay` / `socket.onMsg`), with automatic partner discovery from the first non-admin incoming message. Five message kinds support the coordination protocols: `LLM_UPDATE` propagates each processed instruction from master to slave; `RENDEZVOUS_ARRIVED` signals that an agent has reached the meeting radius, letting a partner already in range complete the mission; and three handoff-specific messages (`HANDOFF_SLAVE_POS`, `HANDOFF_APPROACH`, `HANDOFF_DROPPED`) exchange position and drop-off information during the parcel handoff protocol described below.

Parcel Handoff Protocol. When a `parcel_handoff` command is received, the master assumes the *picker* role and the slave assumes the *deliverer* role. The protocol proceeds in three phases:

1. **Meeting-point negotiation.** The slave reports its position via `HANDOFF_SLAVE_POS`; if received within a configurable timeout, the master runs a BFS-based algorithm (`computeMeetingPoint`) that finds the walkable tile minimizing the summed BFS distance from both agents. Otherwise, the handoff is aborted and the master reverts to normal delivery.
2. **Acquisition and approach.** The picker acquires a parcel via its normal BDI loop, then navigates toward the meeting tile and signals `HANDOFF_APPROACH`. The deliverer

converges on the same tile. Both agents suppress normal delivery intentions during this phase to avoid premature drop-offs.

3. **Drop and pickup.** Once the picker reaches the meeting tile (Manhattan distance ≤ 1), it drops the parcel and broadcasts `HANDOFF_DROPPED` with the drop coordinates; the deliverer's belief state is pre-populated accordingly, so it can immediately pick up and deliver the parcel for the bonus reward. Per-agent timeouts ensure graceful degradation if either agent gets stuck.

3 Experimental Setup

3.1 Scenarios

The agent architecture was evaluated across two main configurations based on the challenge scenarios, specifically designed to test planning efficiency and multi-agent coordination:

- **Single-Agent Benchmarking:** Evaluated on the **first challenge** scenario to isolate and compare planning performance. This setup contrasts the default configuration using Breadth-First Search (BFS) against the proposed PDDL planner.
- **Multi-Agent Coordination:** Evaluated on the **second challenge** scenario to analyze team dynamics. This setup compares a baseline deployment operating without communication against a cooperative team of two agents, where both utilize PDDL planning but their architectures differ: one agent operates solely via BDI, while the other is enhanced with BDI and LLM integration.

To measure the performance and efficiency of each configuration, we consider the **net score** ($R_{\text{net}} = \text{reward} - \text{penalties}$) accumulated by the agents at the end of each map, following a 5-minute run per map.

3.2 LLM Component Evaluation Setup

The LLM component is evaluated in isolation through a dedicated test suite (`tests/llm_tests.ts`), with test cases stored under `tests/data/`. Rather than assessing the system as a monolithic pipeline, the evaluation follows a **component-based approach**, testing each processing stage independently across **282 test cases**.

The dataset covers all major LLM functionalities, including task decomposition, action classification, mathematical expression handling, geographic entity extraction, constraint generation, desire generation, and multi-agent coordination commands. This fine-grained evaluation enables the identification of errors at specific stages of the pipeline and facilitates targeted analysis of failure modes. Ground-truth verification is performed through exact-match checks and structured validation of generated JSON outputs where applicable.

4 Results

Table 1 reports the results on the single agent scenario comparing the **net score** of BFS planning with PDDL using A^* to find the shortest path to reach a tile.

Table 1: Single-agent performance BFS vs PDDL with average net scores.

Map	BFS	PDDL
26c1_1	1416	1310
26c1_2	1630	1582
26c1_3	894	959
26c1_4	1463	1443
26c1_5	681	782
26c1_6	2063	2364
26c1_7	1766	1794
26c1_8	1949	2044
Average	**1482.75**	**1534.75**

The performance variations depend strictly on the structural characteristics of each map. BFS achieves higher net scores on maps that are less constrained and well-connected, where sub-millisecond execution allows the agent to react instantly to fast environmental updates. Conversely, on more complex maps (e.g., `26c1_3` and `26c1_5` through `26c1_8`), the strategic value of optimal action sequences computed by the PDDL planner successfully offsets its call overhead. This structural advantage is reflected in the overall performance, with the PDDL pipeline achieving a higher average net score of 1534.75 compared to the 1482.75 baseline.

Table 2 reports the scores for the multi-agent scenario, where Agent 1 is the pure BDI agent and Agent 2 is the LLM-powered agent.

Table 2: Multi-agent performance: BDI and LLM-powered agent scores.

Map	BDI	LLM-powered	Tested
26c1_1	1069	1262	✓
26c1_2	1098	1031	✓
26c1_3	614	614	✓
26c1_4	529	817	✓
26c1_5	401	530	✓
26c1_6	1697	1041	✓
26c1_7	1136	680	✓
26c1_8	1532	1687	✓

Both agents share the same underlying BDI architecture described in Section 2; the LLM layer acts as an additional reasoning module on top of the reactive loop rather than a replacement. The score differences across maps are primarily driven by the complexity of the natural language instructions provided to the agents: simpler prompts allow the BDI planner to operate with minimal LLM overhead, while more complex directives introduce latency and occasional misclassifications that reduce the effective score. The results, or points, of the prompts tested are not included on the score, as the current server infrastructure can't take them into account by rendering the admin's inputs and, even if it was the case, they wouldn't produce a fair comparison with the pure BDI's points in 1, as it doesn't present an additional source of points.

4.1 LLM Components Evaluation Results

Table 3 reports a component-level evaluation of the LLM pipeline, measuring the accuracy and average latency of each processing stage across all 315 test cases. Given the focus on architectural validation, the metrics were collected on a controlled test set designed to cover the full range of prompt complexity for each category. While the high overall accuracy (98.4%) demonstrates the effectiveness of the targeted few-shot prompts, these figures serve primarily to contextualize execution latency and structural robustness rather than to claim absolute generalization over larger, unconstrained datasets.

Table 3: Component-level evaluation of the LLM pipeline (315 total test cases).

Category	Total	Passed	Avg. latency (ms)
`tools`	50	50	2
`splitMessage`	35	34	764
`extractCityName`	25	25	282
`extractMathExpression`	25	24	372
`extractDeliveryConstraint`	25	25	667
`decideNextAction`	25	25	323
`generateDesire`	23	23	1401
`extractStackConstraint`	8	8	1549
`extractMultiAgentCommand`	22	22	509
`decideNextAction`	44	44	203
Total	**282**	**280**	—

5 Limitations

- **LLM Latency Overhead:** The sequential multi-call pipeline introduces a cumulative latency, acceptable here but unfeasible for real-time, millisecond-range environments.
- **Lack of Competitive Evaluation:** Benchmarks were run without active adversarial agents, so the resilience of the utility functions under dynamic opponents remains untested.
- **Statistical Significance:** Results come from a single run per map; multiple runs would be needed to account for environmental stochasticity.

6 Conclusion

This work presented a full-stack autonomous agent for the Deliveroo.js environment, combining a deliberative BDI loop, a PDDL-based path planner, an LLM natural-language interface, and a cooperative multi-agent protocol.

In the single-agent scenario, the PDDL planner achieved a higher average net score (1534.75) than the BFS baseline (1482.75), confirming that the overhead of optimal path computation is outweighed by the quality of the resulting plans on structurally complex

maps. The BDI loop provided a reliable reactive backbone, with the PDDL layer adding strategic depth where it was most needed.

In the multi-agent scenario the picture is more nuanced. The LLM-powered agent outperforms its pure-BDI counterpart on several maps (e.g., `26c1_1`, `26c1_4`, `26c1_5`, `26c1_8`), demonstrating that natural-language reasoning can translate into actionable decisions that improve parcel collection. However, on maps with faster parcel turnover (e.g., `26c1_6`, `26c1_7`) the cumulative latency of the LLM pipeline erodes this advantage, and the reactive BDI agent scores considerably higher. This trade-off is consistent with the pipeline evaluation, where individual LLM calls range from 2 ms to over 1.5 s depending on task complexity, and with the 98.4% overall accuracy indicating that errors, while rare, do occur under complex prompts.

Taken together, the results confirm that combining symbolic reasoning with natural-language understanding produces a more versatile agent, capable of handling instructions that a purely reactive system could not interpret. The main trade-off is speed: the richer the reasoning, the more time it takes, and fast-paced environments tend to reward quick reactions over deliberation. Closing this gap remains an open direction for future work.

I Prompt Examples

The following are examples of the natural-language prompts used both for the LLM pipeline evaluations and the live benchmarks. Each group corresponds to one processing category in the pipeline.

Message splitting (`splitMessage`) Single-task prompts are forwarded as-is; compound prompts are split into independent sub-tasks before routing.

- *"Go to tile (3, 4) to get +10 points"*
- *"What is 2 + 2?"*
- *"What is the current time in Tokyo?"*
- *"Go to tile (3,4) to get +10pts and tell me the current time in London"*
- *"Calculate 5*3 and tell me the time in Paris"*
- *"Where am I and what time is it in Rome?"*
- *"Go to (2,3) for +5pts, tell me the time in Berlin, and compute sqrt(16)"*
- *"Drop in leftmost tile for +5pts and avoid rightmost tile"*
- *"Deliver stacks of exactly 3 parcels at a time to double the reward"*
- *"Do not go through tile (5, 5) and deliver in (3, 4) for 5x pts"*
- *"Go to (3,4) for +5pts, calculate 10*2, tell me the time in NY, and where am I?"*

Tool / action extraction (`tools`) Maps a raw string to a known tool identifier; handles casing, whitespace, and partial matches.

- *"calculate"*
- *"get_current_time"*
- *"get_my_position"*
- *"generate_desire"*
- *"common_knowledge"*
- *"generate_delivery_constraint"*
- *"CALCULATE"* (uppercase variant)
- *"Generate_Desire"* (mixed case)
- *" calculate "* (leading/trailing whitespace)
- *"GENERATE_DELIVERY_CONSTRAINT"*

City name extraction (`extractCityName`)

- *"What is the current time in London?"*
- *"Tell me the time in Paris"*

- *"What time is it in Tokyo?"*
- *"What is the time in New York?"*
- *"What is the current time in Sydney?"*
- *"I need to know the time in Moscow to schedule a call"*
- *"What time is it in NYC?"*
- *"Che ore sono a Milano?"* (Italian)
- *"What is the time in Amsterdam (CET timezone)?"*
- *"Current time in Seoul please"*

Math expression extraction (`extractMathExpression`)

- *"What is 2 + 2?"*
- *"What is sqrt(144) + 3^2?"*
- *"Compute 15 / 3"*
- *"Evaluate 3 + 4 * 2 - 1"*
- *"What is 2 * pi?"*
- *"Evaluate log(100, 10)"*
- *"What is floor(3.7)?"*
- *"Move to x=4*2 y=(1+3)*3"* (embedded in navigation command)
- *"Calculate min(10, 20, 5)"*
- *"What is 10 % 3?"*
- *"Compute sin(0)"*
- *"Calculate 1000 * 1000"*

Delivery tile constraint (`extractDeliveryConstraint`)

- *"Drop packages in leftmost tile to get +5 points"*
- *"Deliver to rightmost tile for +10 points"*
- *"Drop in topmost tile to earn 8 points"*
- *"You lose 5 points if you drop in the bottommost tile"*
- *"Never deliver to the topmost tile"*
- *"Avoid dropping in the leftmost tile"*
- *"The rightmost tile is forbidden for dropping"*
- *"Get +50 pts by dropping at leftmost"*
- *"Rightmost tile gives -8 pts penalty"*

- *"Consegna sulla casella più a sinistra per +5 punti"* (Italian)

Stack constraint (`extractStackConstraint`)

- *"Deliver stacks of exactly 3 parcels at a time to double the reward"*
- *"Deliver stacks of exactly 5 parcels at a time to get 0.3 of the standard reward"*
- *"Delivering 4 or more parcels at once gives you 1.5x the reward"*
- *"Delivering at most 2 parcels cuts the reward in half"*
- *"Delivering exactly 1 parcel gives you no reward"*
- *"Delivering 2 or more parcels gives 1.2x the normal reward"*
- *"Exactly 5 parcels at once gives 0.3 of the standard reward"*
- *"Consegna esattamente 3 pacchi per ottenere il doppio del reward"* (Italian)

Desire generation (`generateDesire`) Generates a BDI desire (go-to, avoidance, or delivery multiplier) from a spatial instruction.

- *"Go to tile (4, 5) for +10 pts"*
- *"Avoid tile (1, 1), penalty -20pts"*
- *"Do not go through tile (2, 3) otherwise you lose 50pts"*
- *"Tile (5, 5) costs you 50 points if you walk through it"*
- *"Steer clear of (7, 1) or you lose 50 points"*
- *"Every time you deliver in (3, 4) you get 5x pts"*
- *"Delivering at (0, 7) gives you 3x the normal reward"*
- *"Every time you deliver in (1, 2) you get 0 pts"*
- *"delivery tile 6,13 gives x0 points"*
- *"delivery tile 4,7 gives x0.5 points"*
- *"delivery tile 2,13 gives x200 of the reward"*
- *"delivering to tile (2, 8) gives you 0.3 of the standard reward"*
- *"deliverying in 2,13 gives triple the reward"*
- *"Vai alla casella (6, 2) per +15 punti"* (Italian)
- *"Non passare per la casella (2, 2), perdi 50 punti"* (Italian)

Plan task parsing (`plan_tasks`) Resolves arithmetic expressions embedded in tile coordinates or reward values before execution.

- *"Go to tile (3, 4) to get some points"*
- *"Move to x=4*2 to get +10pts"*
- *"Move to x=4*2 y=(1+3)*3 to get +10pts"*

- *"Go to tile ((5+3)/2, 4)"*
- *"Move to x=sqrt(16) y=4"*
- *"Go to tile (floor(7.5), 3)"*
- *"Go to x=round(pi) y=5"*
- *"Move to x=abs(-3) y=ceil(2.1)"*
- *"Go to tile (2^3, 2^2)"*
- *"Vai a x=3*4 y=2 per +10 punti"* (Italian)

Next action routing (`decideNextAction`) Routes an incoming message to the correct handler: math, time, position, desire, delivery constraint, stack constraint, or general knowledge.

- *"What is sqrt(144) + 3^2?"* (math)
- *"What is the current time in London?"* (time)
- *"Where am I?"* (position)
- *"Go to tile (3, 4) for +10 points"* (desire)
- *"Avoid tile (9, 9) penalty -20pts"* (desire / avoidance)
- *"Drop packages in the leftmost tile for +5 points"* (delivery constraint)
- *"Deliver stacks of exactly 3 parcels at a time to double the reward"* (stack constraint)
- *"delivery tile 2,13 gives x200 of the reward"* (delivery multiplier)
- *"What is the capital of France?"* (general knowledge)
- *"Who invented the telephone?"* (general knowledge)
- *"Dove sono adesso?"* (Italian — position)
- *"Vai alla casella (5, 3) per +10 punti"* (Italian — desire)

Multi-agent commands (`extractMultiAgentCommand`) Covers stop/resume signals, row-based positioning, rendezvous coordination, and parcel handoff bonuses.

- *"Red light!"* (stop all agents)
- *"Red light — all agents stop on an odd row"*
- *"Red light — stop on an even row"*
- *"Stop! All agents freeze immediately."*
- *"All agents halt and hold position."*
- *"Agents must stand still on an even row."*
- *"All agents must move to an odd-numbered row and wait"*
- *"Green light."* (resume all agents)

- *"Green light, go!"*
- *"Resume moving."*
- *"Continue your deliveries."*
- *"Move both agents to the neighborhood of position (5,3) within a maximum distance of 3 and have them wait for each other. You will receive 500pts."* (rendezvous)
- *"Move both agents to the neighborhood of position 4,20 within a maximum distance of 3 and have them wait for each other"* (rendezvous)
- *"If a parcel is initially picked up by one agent and later delivered by the other agent, you will receive a 200 points bonus."* (handoff)
- *"You get 500 extra points if one agent picks up a parcel and the partner delivers it."* (handoff)
- *"Pass the package to the other agent for a 50 pt reward."* (handoff)
- *"Relay the delivery! Have agent 1 transfer the box to agent 2."* (handoff)